High-performance Computing Coursework Report

1. Unit Tests

Unit tests verify whether certain part of the codebase works as expected by testing it with various input scenarios and asserting that the output matches the expected result. General information of unit tests implemented is summarized in Table 1.

Table 1. Information of unit tests

Class	*LidDrivenCavity*	*SolverCG*
Test file name	Lid_test.cpp	SolverCG_test.cpp
Number of unit tests	1	4
Commit number	6fe0102	
Run command	make test	

1.1 LidDrivenCavity class

One unit test was implemented using the BOOST_TEST_MODULE, named Lid_test.cpp, to test the correct initialization of class *LidDriveCavity*.

The parameters being tested are summarized in Table 2:

Table 2. Parameters tested in *LidDriveCavity* class

Methods	Quanity
lid.setDomainSize	*Lx*=1.0, *Ly*=1.0
lid.SetGridSize	*Nx*=11, *Ny*=11
lid.SetTimeStep	*dt*=0.01, *T*=1.0
lid.SetReynoldsNumber	*Re*=100, *nu*=0.01
lid.Npts	*Npts*=121
Allocation of arrays	*v*, *s*, temp, cg

1.2 ***SolverCG* class**

Four unit tests are implemented for the *SolverCG* class, named SolverCG_test.cpp, and are summarized in Table 3.

Table 3. Summary on unit tests for *SolverCG* class

Name of the unit test	The method in *SolverCG* being tested	Description
testConstructor	*Constructor*	Verifies that the *SolverCG* class is initialized correctly with the provided *Nx*, *Ny*, *dx* and *dy* values
testApplyOperator	*ApplyOperator*	Checks if the finite difference operator is correctly applied and produces the expected output
testPreconditioner	*Preconditioner*	Checks if outputs correct scaling factor and interior point
testImposeBC	*ImposeBC*	Checks if the boundary conditions are correctly imposed

The logic to verify the subsequent functions are giving a correct output is described as the following:

By defining *testDx* = *testDy* = 1.0, and a 3×3 input matrix:

$$testIn = \begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{matrix} \quad (1)$$

and applies the finite difference operator with respect to the macro IDX used in the *ApplyOperator* function, the expected output should be obtained as:

$$expectedApplyOperator = \begin{matrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{matrix} \quad (2)$$

Furthermore, by applying:

$$factor = 2 * (1/dx^2 + 1/dy^2) = 4 \quad (3)$$

$$\text{out[i, j] = in[i, j] / factor} \quad \text{(for interior points)} \quad (4)$$

$$\text{out[i, j] = in[i, j]} \quad \text{(for boundary points)} \quad (5)$$

the test code should output:

$$expectedPrecndition = \begin{matrix} 1 & 2 & 3 \\ 4 & 1.25 & 6 \\ 7 & 8 & 9 \end{matrix} \quad (6)$$

Finally, the *ImposeBC* should return a matrix of:

$$expectedImposeBC = \begin{matrix} 0 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 0 \end{matrix} \quad (7)$$

2. MPI

2.1 Parallelization approach

1. The code divides the computational domain into subdomains by creating cartesian topology. Each subdomain is handled by a separate MPI process.
2. To ensure the work done by each process is as balanced as possible, the domain is divided by calculating the sub-grid size based on the process's coordinates in the Cartesian grid. Special consideration was given to distributing any extra rows or columns that don't evenly divide among the processes, in order to balance the computational load.
3. The conjugate gradient solver was parallelized by ensuring the operations, including dot products and vector updates are carried out in parallel across all processes. MPI collective operations are used to get global results.
4. *Communicator.cpp* and its header file are introduced to manage communication between the rank's boundaries, via *ExchangBoundaryData*.

A series of scaling tests were conducted, for Re=1000, T=50, on a 1x1 2D grid. Each individual test's parameters and computational time are summarized in Table 4. The corresponding scaling plots showing the speedup are shown in Figure 1.

Table 4. MPI scaling tests summary

Test No.	1	2	3	4	5
Number of processors	Nx=Ny=21	Nx=Ny=63	Nx=Ny=101	Nx=Ny=121	Nx=Ny=121
dt	0.005	0.005	0.005	0.005	0.01
1	2.7354	92.4406	367.698	582.097	299.278
4	4.3343	37.9086	143.425	202.949	104.839
9	7.7904	38.7534	93.1884	149.242	72.171
16	9.5214	38.8656	69.6306	95.364	54.6959

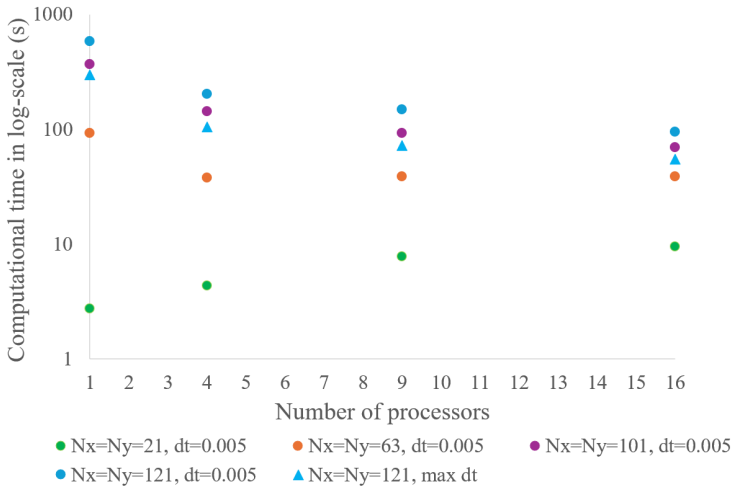


Figure 1. MPI scaling plot: computational time vs number of processors

A list of conclusions can be drawn below:

1. It was observed from Figure 1 that for a relatively small job size, Nx=Ny=21, dt=0.005, the serial processing is the fastest among all other parallelized ones. Achieving optimal performance requires ensuring that all processors always have approximately equal amounts of work, this is guaranteed by $P=p^2$, which decomposes the domain into p ways in each dimension. However, a rule of thumb is that one should always maximize the utilization of each processor according to each individual job size.
2. Similarly, for Nx=Ny=63, dt=0.005, the optimal number of processors is 4
3. For the rest of test cases, a strong scaling is observed when changing from serial computing to using four processors, and a relatively shallower decrease in computational time when using more than 4 processors. Use Mystery-Cluster 16 GPU stations as an example, there are four nodes with four GPUs on each node. As a result, when using more than 4 GPUs, the communication time increases as cross-node communication would take place, which results in a shallower increase in performance. Similarly, as the MPI number of processes increases, so does the overhead associated with communication between the processes. Especially for a problem which requires frequent and extensive data exchange, the overhead could diminish the benefits of parallelization.

Furthermore, as Amdahl's Law highlights that the speedup of a program using multiple processors in parallel computing is limited by the time needed for the sequential fraction of the program. Therefore, even with an infinite number of processors, the speedup is constrained by the serial portion of the program.

4. As the lecturer stated on *Ed Forum*, the program would be killed due to memory leak, which could be due to various reasons, including deficient MPI algorithm implemented, exceeding Quotas, and even system overload due to heavy usage by the whole cohort. Unfortunately, the maximum job size tested is Test 4, Nx=Ny=121, dt=0.005 for 50 convective time units in total.
5. In addition, Test 5 was tested out with a dt = 0.01, twice as big as Test 4's, which achieved a twice reduction in computational time. Accordingly, while satisfying the time step restriction for stability in *CFD* problems, one could use the biggest dt allowed to save computational cost.
6. As summarized above, for a sufficient job size, the scaling meets expectation.

3. OpenMP

3.1 Parallelizing approach

To add multithread to the MPI-based code using OpenMP, I started by identifying loops or sections where iterations are independent and can be simultaneously executed without affecting the outcome. The changes include: *ApplyOperator*, *Precondition* and conjugate gradient *Solver* void in *SolverCG.cpp*, *BoundaryV* and *ExchangeBoundaryData* void in *LidDrivenCavity.cpp*.

3.2 Scaling tests

Unfortunately, running with multi-thread on multiple processes is diverge for now. A series of scaling tests have been solely run on serial, with parameters: Re=1000, dt=0.005, T=50, Nx=Ny=101. The scaling results are shown in Figure 2.

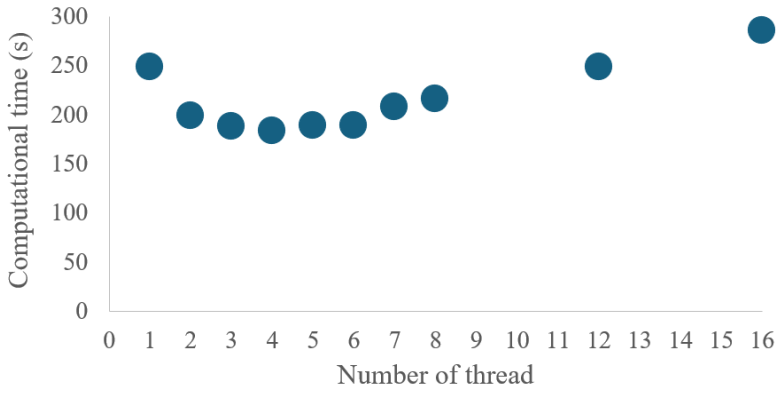


Figure 2. OpenMP scaling tests run on serial

It's observed that the optimal number of threads is 4, which completes the computation quickest. Then the computational gradually increases, and it finishes the slowest with the greatest number of threads 16. Since the clear trend with a global minimum has been seen, only thread 12 and 16 are tested after tested with 8 threads.

As the number of threads increases, they may compete for access to memory, leading to contention and saturation of the memory bandwidth. This is especially true for memory-bound applications where the bottleneck is the rate at which data can be read from or written to memory, rather than the speed of computation. Furthermore, increased synchronization needs can arise

with more threads, especially if threads need to communicate frequently or wait on shared resources. The overhead from managing these synchronization primitives can outweigh the benefits of parallel execution. Overhead associated with creating, managing, and scheduling more threads by the operating system can also contribute to decreased performance.